

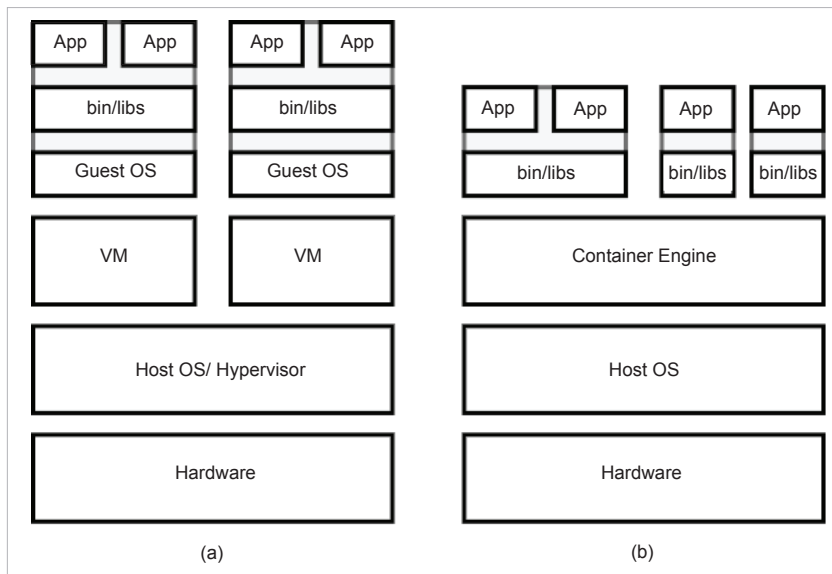


Figure 1: (a) Multiple VMs over the top of the hardware
(b) Multiple containers are mounted over the host OS without any additional guest OS or hypervisor

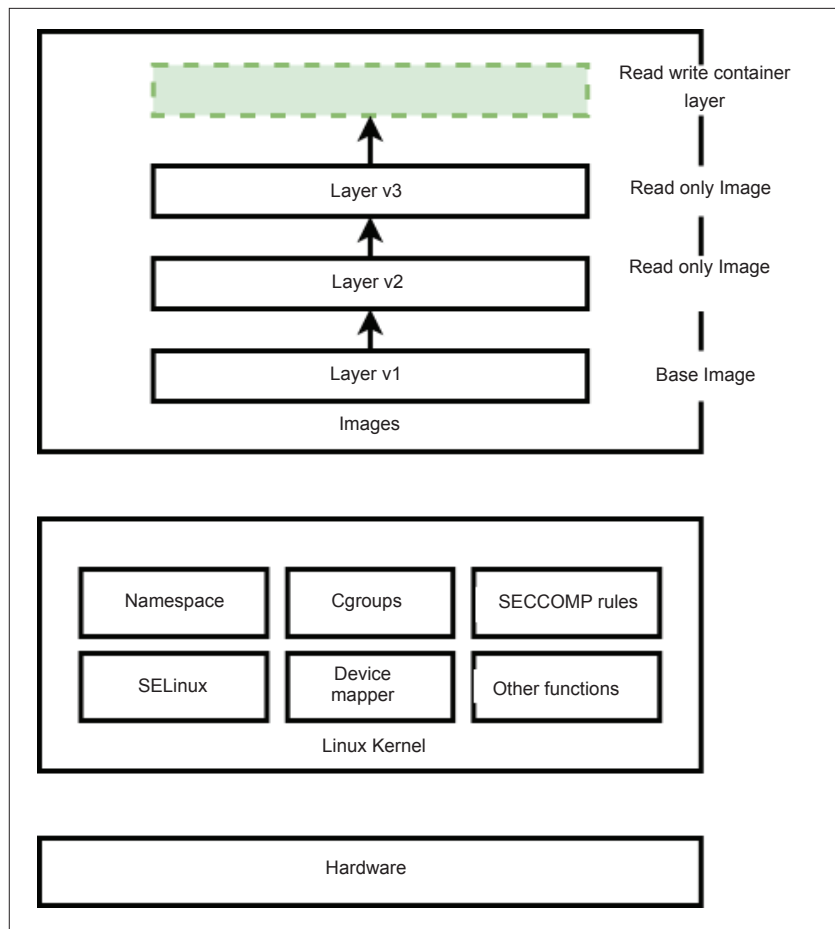


Figure 2: Various image layers over the top of container architecture

network interface. The cgroup feature takes care of the allocation and control of system resources for a group. The cgroup has a resource controller that monitors as well as accounts for resource usage. It is used by many of its subsystems such as debug, devices, freezer, perf-event, CPU, cpuset, cpuacct, memory, hugetlb, blkio, net-cls, and systemd. These subsystems can be viewed in the `/sys/fs/cgroup` directory. The process ID inside the group is global; therefore, it is NOT isolated from the host. However, the requirement here is to isolate each group.

The name space feature helps the group to isolate its process IDs from the host system. The isolation features are implemented in the mounted file systems, UTS, IPC, PID, network, and user. This way, each group is created as an isolated environment for a user with the help of cgroup and name space.

The container runs as a user process inside the server and helps make every operation possible, as is done by a user process such as creation, deletion, monitoring, performance tuning, etc. At the same time, it is isolated from other system processes on the server. Therefore, each container works like a standalone system. The container can have its own process IDs, mounted file systems, system resources, network, and logs. A container can be executed without adding any overhead to the hypervisor, and launching a container is miraculously fast.

Figure 2 depicts how various layers are added to the top of the base image of the container, where the container is connected directly to the Linux kernel via the daemon process running over the host operating system. The top layer of the image can always be topped with another layer, where the user can add self-defined functionalities.